

Conceitos de Qualidade de Software

O que é Qualidade?

Qualidade é um conceito complexo e multifacetado que pode ser difícil de definir, mas é reconhecido quando observado. Para software, qualidade precisa ser definida explicitamente. David Garvin sugere cinco visões de qualidade:

1. **Transcendental:** Qualidade é algo reconhecido intuitivamente, mas difícil de definir.
2. **Do usuário:** Qualidade é medida pelo atendimento às metas específicas do usuário final.
3. **Do fabricante:** Qualidade é definida pelo cumprimento das especificações originais do produto.
4. **Do produto:** Qualidade está ligada às características inerentes do produto, como funções e recursos.
5. **Baseada em valor:** Qualidade é medida pelo valor que o cliente está disposto a pagar.

No desenvolvimento de software, qualidade envolve tanto o atendimento às especificações de projeto quanto a conformidade na implementação, garantindo que o sistema atenda às necessidades e metas de desempenho.

O que é qualidade de software?

A qualidade de software é definida como uma gestão de qualidade efetiva que cria um produto útil, fornecendo valor mensurável para quem o produz e utiliza. Ela envolve três pontos principais:

1. **Gestão de qualidade efetiva:** Estabelece uma infraestrutura que evita o caos no projeto, promove práticas de engenharia de software consistentes e inclui atividades de apoio como gerenciamento de mudanças e revisões técnicas.
2. **Produto útil:** Deve atender às exigências explícitas e implícitas dos usuários, como confiabilidade e facilidade de uso.
3. **Valor agregado:** Beneficia tanto o fabricante, reduzindo custos de manutenção e suporte, quanto os usuários, otimizando processos de negócios e aumentando a rentabilidade.

Dimensões de qualidade de Garyin

David Garvin propõe oito dimensões de qualidade aplicáveis ao software, como desempenho, confiabilidade, conformidade, durabilidade, facilidade de manutenção, estética e percepção. Além disso, fatores de qualidade como os de McCall e ISO 9126 ajudam a avaliar aspectos como funcionalidade, usabilidade,

eficiência e portabilidade. Embora muitos desses fatores sejam subjetivos, eles fornecem uma base para medir e melhorar a qualidade do software.

As dimensões de qualidade de Garvin são:

1. **Qualidade do desempenho:** O software fornece todas as funções e recursos especificados no modelo de requisitos, gerando valor ao usuário final?
2. **Qualidade dos recursos:** O software surpreende e encanta os usuários finais ao utilizá-lo pela primeira vez?
3. **Confiabilidade:** O software funciona sem falhas, está disponível quando necessário e fornece funcionalidade sem erros?
4. **Conformidade:** O software está de acordo com padrões locais e externos, além de seguir convenções de projeto e codificação?
5. **Durabilidade:** O software pode ser mantido ou corrigido sem gerar efeitos colaterais indesejados?
6. **Facilidade de manutenção:** O software pode ser modificado ou corrigido em um tempo aceitável, com informações acessíveis para suporte?
7. **Estética:** O software apresenta elegância, fluidez e uma "presença" que são subjetivas, mas evidentes.
8. **Percepção:** A qualidade percebida pode ser influenciada por preconceitos, como a reputação do fornecedor.

Essas dimensões fornecem uma visão abrangente e subjetiva da qualidade de software.

Fatores de qualidade de McCall

Os fatores de qualidade de McCall, Richards e Walters categorizam aspectos que afetam a qualidade de software em três áreas principais: características operacionais, habilidade de suportar mudanças e adaptabilidade a novos ambientes. Os fatores incluem:

1. **Correção:** O software atende às especificações e objetivos do cliente.
2. **Confiabilidade:** Realiza funções pretendidas com precisão esperada.
3. **Eficiência:** Utiliza recursos computacionais de forma otimizada.
4. **Integridade:** Controla o acesso não autorizado ao software ou dados.
5. **Usabilidade:** Facilita o aprendizado, operação e interpretação de resultados.
6. **Facilidade de manutenção:** Simplifica a localização e correção de erros.
7. **Flexibilidade:** Permite modificações com esforço aceitável.
8. **Testabilidade:** Facilita a verificação de que o software cumpre suas funções.
9. **Portabilidade:** Facilita a transferência para outros ambientes.
10. **Reusabilidade:** Permite reutilização em outras aplicações.
11. **Interoperabilidade:** Facilita a integração com outros sistemas.

Embora muitos desses fatores sejam medidos indiretamente, eles fornecem uma base sólida para avaliar a qualidade de software.

Fatores de qualidade ISO 9126

O padrão ISO 9126 identifica seis atributos fundamentais de qualidade para software, cada um com subatributos que ajudam na avaliação:

1. **Funcionalidade:** Mede o quanto o software atende às necessidades declaradas, incluindo adequabilidade, exatidão, interoperabilidade, conformidade e segurança.
2. **Confiabilidade:** Avalia o tempo de disponibilidade do software, considerando maturidade, tolerância a falhas e facilidade de recuperação.
3. **Usabilidade:** Analisa a facilidade de uso do software, incluindo compreensão, aprendizagem e operabilidade.
4. **Eficiência:** Examina o uso otimizado dos recursos do sistema, como comportamento em relação ao tempo e aos recursos.
5. **Facilidade de manutenção:** Avalia a facilidade de realizar correções, considerando análise, realização de mudanças, estabilidade e testabilidade.
6. **Portabilidade:** Mede a facilidade de transferir o software para outros ambientes, incluindo adaptabilidade, instalação, conformidade e substituição.

Embora esses fatores não levem necessariamente à medição direta, eles fornecem uma base para medidas indiretas e uma lista de verificação útil para avaliar a qualidade de um sistema.

Fatores de Qualidade Desejados

Os fatores de qualidade desejados complementam as dimensões e fatores gerais de qualidade, focando em características específicas e mensuráveis para avaliar a qualidade de software. Exemplos incluem:

1. **Intuição:** Avalia se a interface segue padrões esperados, facilitando o uso por novatos sem treinamento significativo.
2. **Eficiência:** Mede a facilidade de localizar e realizar operações, minimizando movimentos e apresentando dados de forma compreensível.
3. **Robustez:** Examina como o software lida com dados incorretos ou interações inadequadas, garantindo operação contínua e diagnósticos úteis.
4. **Riqueza:** Analisa se a interface oferece recursos importantes, como personalização e macros para simplificar operações.

Esses fatores são avaliados por meio de perguntas práticas que ajudam a determinar se o software atende aos requisitos de qualidade. Um conjunto similar de perguntas pode ser desenvolvido para outros fatores de qualidade.

Software “bom o suficiente”.

O conceito de software "bom o suficiente" refere-se à entrega de um produto que atende às principais funções e características desejadas pelos usuários, mesmo contendo erros conhecidos em funcionalidades menos relevantes. Essa abordagem é adotada por grandes empresas que priorizam o tempo de colocação no mercado, planejando melhorias para versões futuras.

Embora essa filosofia funcione em alguns casos, ela apresenta riscos significativos:

- **Reputação:** Pequenas empresas podem sofrer danos irreparáveis à sua imagem ao entregar produtos com baixa qualidade.
- **Domínios críticos:** Em áreas como software embarcado ou integrado com hardware, erros podem levar a litígios, prejuízos financeiros e até consequências legais.
- **Confiança do cliente:** Software com falhas pode comprometer a satisfação e fidelidade dos usuários.

Portanto, a ideia de "bom o suficiente" deve ser aplicada com cautela, considerando os impactos no mercado e nos domínios de aplicação.

Custo da Qualidade

O custo da qualidade engloba os gastos necessários para garantir a qualidade de software e os custos gerados pela falta dela. Ele é dividido em três categorias:

1. **Custos de prevenção:** Incluem atividades como planejamento, desenvolvimento de modelos completos, treinamento e testes para evitar defeitos.
2. **Custos de avaliação:** Relacionam-se à análise da qualidade dos produtos, como revisões técnicas, coleta de métricas e testes.
3. **Custos de falhas:** Abrangem os gastos com erros detectados antes (falhas internas) ou após a entrega (falhas externas), como retrabalhos, suporte ao cliente e perda de reputação.

Os custos aumentam significativamente à medida que os erros são detectados em fases mais avançadas do ciclo de desenvolvimento. Investir em prevenção e controle inicial é essencial para reduzir os gastos totais e evitar problemas futuros.

Riscos – Um caso real

Os riscos associados à baixa qualidade de software podem impactar tanto os desenvolvedores quanto os usuários finais. Além de custos elevados, software mal projetado pode levar a consequências graves, como falhas em sistemas críticos. Um exemplo extremo é o caso de um hospital no Panamá, onde

modificações inadequadas em software resultaram em doses excessivas de radiação, causando mortes e complicações graves.

Esse caso destaca que software de baixa qualidade pode gerar riscos significativos, incluindo litígios, danos à reputação e até consequências fatais. A qualidade de software é essencial para evitar esses problemas e garantir segurança e confiabilidade em aplicações críticas.

Alcançando a Qualidade Software

A qualidade de software é resultado de bom gerenciamento de projetos e práticas consistentes de engenharia. Quatro atividades principais são essenciais:

1. **Métodos de engenharia de software:** Compreender o problema e criar projetos adequados que atendam às dimensões e fatores de qualidade.
2. **Técnicas de gerenciamento de projeto:** Garantir estimativas realistas, cronogramas bem planejados e gestão de riscos para evitar caos e manter a qualidade.
3. **Controle de qualidade:** Realizar revisões, inspeções e testes para identificar e corrigir erros, ajustando o processo conforme necessário.
4. **Garantia da qualidade:** Estabelecer uma infraestrutura que suporte práticas sólidas, auditorias e relatórios para monitorar e melhorar a qualidade.

Essas atividades, quando aplicadas de forma integrada, ajudam a construir software de alta qualidade, atendendo às expectativas dos usuários e aos objetivos do projeto.

Referência

Engenharia de Software - Uma Abordagem Profissional

Roger S. Pressman